

JAVA STRING HANDLING AND COLLECTION FRAMEWORK

Hashing and Collisions in HashMap — Analysis & Solution

Name : M.Yogeshwaran

Reg No : 192524363

Course Name : Java Programming

Topic: Hashing and Collisions

1. Aim

To analyse the likely reasons why performance degrades when millions of subscriber records are stored in a HashMap due to many keys falling into the same hash buckets, and to design and implement a corrective Java program that demonstrates the difference between a poorly hashed key and a well-designed key.

2. Problem Analysis

2.1 Likely reasons for the performance degradation:

- Poor hashCode() implementation on the key class — if hashCode() returns a constant or a value derived from very few bits (e.g. only the area code of a mobile number), a large number of distinct keys map to the same bucket.
- Overridden equals() without a matching hashCode() (or vice-versa) — breaks the hashCode/equals contract, causing incorrect bucket lookup and unnecessary chaining.
- Using mutable objects as keys — if a field used in hashCode() changes after insertion, the entry becomes "lost" in the wrong bucket.
- Low initial capacity / default load factor for a very large dataset — frequent resizing (rehashing) of millions of entries is expensive and temporarily increases collisions.
- Skewed real-world key distribution — subscriber IDs or mobile numbers that share common prefixes (e.g. same STD/area code) can cluster into fewer buckets if hashCode() does not mix the bits well.
- Using non-final / mutable String-like keys built by concatenation without proper hashCode caching, leading to repeated and inconsistent hash computation.
- Bucket lists degenerating to O(n) traversal per bucket — from Java 8 onwards a bucket is treeified (O(log n)) only when it holds 8 or more entries AND the key implements Comparable; custom keys that are not Comparable stay as O(n) linked lists.

2.2 Recommended corrective actions:

- Always override hashCode() and equals() together, using all significant, immutable fields (java.util.Objects.hash() is a safe default).
- Use immutable key classes (final fields, no setters) so the hash code never changes after insertion.
- Pre-size the HashMap using the expected number of entries: new HashMap<>(expectedSize / 0.75f) to avoid repeated rehashing for millions of records.
- Choose a good load factor (0.75 is usually optimal — balances memory use and lookup speed).
- Prefer well-tested key types (String, Long, wrapper classes, records) over custom classes with hand-rolled poor hash logic.
- Make custom keys implement Comparable so Java 8+ can treeify overloaded buckets (O(log n) instead of O(n)).
- For extremely large or concurrent datasets, consider ConcurrentHashMap (better bucket-level concurrency) or partitioning data across multiple maps/shards.

- Monitor bucket distribution / collision counts in testing and profile with realistic production-like key samples before going live.

3. Algorithm / Pseudocode

1. Start
2. Define a Subscriber class to hold record data (id, name, plan).
3. Define BadKey class whose hashCode() always returns the same constant value (simulates a poorly designed key).
4. Define GoodKey class whose hashCode() combines the key's fields using Objects.hash() (simulates a well designed key).
5. Create badMap (HashMap<BadKey,Subscriber>) and goodMap (HashMap<GoodKey,Subscriber>), pre-sizing goodMap.
6. Insert n subscriber records into badMap using BadKey objects; record the time taken.
7. Insert n subscriber records into goodMap using GoodKey objects; record the time taken.
8. Perform n lookups on badMap using new BadKey objects; record the time taken.
9. Perform n lookups on goodMap using new GoodKey objects; record the time taken.
10. Print insertion time, lookup time and map sizes for both maps.
11. Compare the two results and conclude which key design performs better and why.
12. Stop

4. Java Program

```
import java.util.*;

public class HashCollisionDemo {

    // ---- Subscriber record stored as the HashMap value ----
    static class Subscriber {
        String id, name, plan;
        Subscriber(String id, String name, String plan) {
            this.id = id; this.name = name; this.plan = plan;
        }
        public String toString() {
            return id + " - " + name + " (" + plan + ")";
        }
    }

    // ---- BAD KEY : deliberately poor hashCode() ----
    // Every object returns the SAME hash code, so every entry
    // lands in a single bucket (worst-case linked list).
    static class BadKey {
        final String mobile;
        BadKey(String mobile) { this.mobile = mobile; }

        @Override
```

```

    public int hashCode() {
        return 1; // <-- root cause of collisions
    }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof BadKey)) return false;
        return mobile.equals(((BadKey) o).mobile);
    }
}

// ---- GOOD KEY : well distributed hashCode() ----
// Uses Objects.hash() on an immutable field so bits are
// spread evenly across buckets.
static class GoodKey {
    final String mobile;
    GoodKey(String mobile) { this.mobile = mobile; }

    @Override
    public int hashCode() {
        return Objects.hash(mobile); // <-- good distribution
    }
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof GoodKey)) return false;
        return mobile.equals(((GoodKey) o).mobile);
    }
}

public static void main(String[] args) {

    int n = 20000; // subscriber count

    Map<BadKey, Subscriber> badMap = new HashMap<>();
    Map<GoodKey, Subscriber> goodMap = new HashMap<>(n * 2); // pre-sized

    // ----- Insert using BadKey -----
    long t1 = System.nanoTime();
    for (int i = 0; i < n; i++) {
        String mobile = "9" + String.format("%09d", i);
        badMap.put(new BadKey(mobile),
            new Subscriber("SUB" + i, "User" + i, "PLAN-A"));
    }
    long t2 = System.nanoTime();

    // ----- Insert using GoodKey -----
    for (int i = 0; i < n; i++) {
        String mobile = "9" + String.format("%09d", i);
        goodMap.put(new GoodKey(mobile),

```

```

        new Subscriber("SUB" + i, "User" + i, "PLAN-A"));
    }
    long t3 = System.nanoTime();

    // ----- Lookup using BadKey -----
    for (int i = 0; i < n; i++) {
        String mobile = "9" + String.format("%09d", i);
        badMap.get(new BadKey(mobile));
    }
    long t4 = System.nanoTime();

    // ----- Lookup using GoodKey -----
    for (int i = 0; i < n; i++) {
        String mobile = "9" + String.format("%09d", i);
        goodMap.get(new GoodKey(mobile));
    }
    long t5 = System.nanoTime();

    System.out.println("Number of subscriber records : " + n);

System.out.println("-----");
    System.out.printf("Insertion time (BadKey - all collide) : %d ms%n",
(t2 - t1) / 1_000_000);
    System.out.printf("Insertion time (GoodKey - well spread) : %d ms%n",
(t3 - t2) / 1_000_000);
    System.out.printf("Lookup time      (BadKey - all collide) : %d ms%n",
(t4 - t3) / 1_000_000);
    System.out.printf("Lookup time      (GoodKey - well spread) : %d ms%n",
(t5 - t4) / 1_000_000);

System.out.println("-----");
    System.out.println("BadMap size : " + badMap.size());
    System.out.println("GoodMap size : " + goodMap.size());
}
}

```

5. Output

```

Number of subscriber records : 20000
-----
Insertion time (BadKey - all collide) : 2840 ms
Insertion time (GoodKey - well spread) : 9 ms
Lookup time      (BadKey - all collide) : 2793 ms
Lookup time      (GoodKey - well spread) : 6 ms
-----
BadMap size : 20000
GoodMap size : 20000

```

Note: exact timings vary with JVM version, JIT warm-up and machine load, but the BadKey run is consistently and dramatically slower than the GoodKey run at the same record count.

6. Result

The program clearly demonstrates the root cause of the reported performance degradation. When the key's `hashCode()` returns the same value for every object (BadKey), all 20,000 records are forced into a single bucket. Since a plain (non-Comparable) custom key cannot be treeified by the JVM, that bucket becomes a single linked list, so every `put()` and `get()` operation must scan up to n entries — turning average-case $O(1)$ HashMap behaviour into worst-case $O(n)$ per operation and $O(n^2)$ overall. This matches exactly what was observed in the telecom system once millions of subscriber records were loaded: insertion and lookup times grew quadratically instead of remaining near-constant.

With a properly designed key (GoodKey) that spreads hash codes evenly using `Objects.hash()`, and with the map pre-sized to the expected record count, both insertion and lookup complete in near-constant amortised time, exactly as a HashMap is expected to behave. Therefore, correcting the `hashCode()/equals()` implementation, using immutable keys, pre-sizing the map, and (where applicable) making keys Comparable are effective and verified solutions to the hash-collision performance problem.